

An Efficient and Lightweight Hallucination Detection Framework for RAG Using SLMs

By

Mehrin Sultana – 0112230928

Mominul Islam – 0112230425

Nazmul Alam – 0112230914

Md. Mainul Hossain Fahim Chowdhury – 0112230085

**Supervised By Assistant Professor Ms. Khushnur Binte
Jahangir**

Submitted in partial fulfilment of the requirements
of the degree of Bachelor of Science in Computer Science and Engineering

April 5, 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
UNITED INTERNATIONAL UNIVERSITY

Contents

List of Tables	3
1 Introduction	4
1.1 Project Overview	4
1.2 Motivation	4
1.3 Objectives	5
1.4 Methodology	6
1.5 Organization of the Report	6
2 Background	8
2.1 Preliminaries	8
2.1.1 Large Language Models(LLM)	8
2.1.2 Retrieval-Augmented Generation (RAG)	8
2.1.3 Hallucinations in Large Language Models (LLMs)	9
2.1.4 Approaches for Detecting Hallucinations	9
2.1.5 Key Algorithms and Architectures	10
2.1.6 Evaluation Metrics	10
2.2 Literature Review	11
2.2.1 Similar Applications (Evaluation Frameworks)	11
2.2.2 Related Research Papers	11
2.2.3 Gap Analysis	13
3 Project Design	14
3.1 Requirement Analysis	14
3.1.1 Functional and Nonfunctional Requirements	14
3.1.2 Context Diagram	15
3.1.3 Data Flow Diagram	15
3.1.4 Design	15
3.2 Detailed Methodology and Design	17
3.3 Project Plan	17
3.4 Task Allocation	18
3.5 Summary	19
5 Standards and Design Constraints	20
5.1 Complex Engineering Problems Activities	20
5.1.1 P's that are addressed through our project	20

Contents	2
5.4 Summary	23
References	24

List of Tables

2.1	Gap Analysis	13
3.1	Task Allocation (SLM Hallucination Detector – Group B2)	18
3.1	Task Allocation (SLM Hallucination Detector – Group B2)	19
5.1	Mapping with complex problem solving.	21
5.2	Project Cost Analysis	22

Chapter 1 Introduction

In this section, we will get to know the reason behind our work and how we are going to achieve it.

1.1 Project Overview

Retrieval-Augmented Generation(RAG) systems help make Large Language Models(LLM) give accurate answers [13]. They do this by pulling in information and using that to create responses. With this help these systems can still make up information that isn't fully backed up by the facts they've gathered. Earlier language models only used what they already knew to answer questions. This often led to wrong answers. To fix this Retrieval-Augmented Generation(RAG) was created to bring in knowledge. Even with this update the systems can still make mistakes. This is a deal in areas like education, healthcare [8] and search systems. People rely on these systems to give them information [11]. If the answers are wrong it can spread misinformation, make users lose trust and even cause problems in areas. Detecting when these systems make up information is hard. Most methods rely on language models that use a lot of computer power and are expensive. They also aren't good for real-time use. We need a way to spot made-up information that doesn't use a lot of computer power. So the goal of this research is to create a system that can spot made-up information in Retrieval-Augmented Generation(RAG) outputs. This system should be able to work even with small language models and not use too much computer power.

1.2 Motivation

With Large Language Models(LLM) getting fast, RAG-based systems are being used more and more in things like question answering, chatbots and knowledge retrieval systems [5, 8]. These systems still sometimes make up answers, which makes them less reliable and less used in real-life situations.

To make AI systems trustworthy we need to make sure they give answers [11]. However, using models to check for made-up answers can be too expensive and use too much computer power especially when resources are limited. So we need an efficient way to detect made-up answers without using expensive models. Some challenges in this area are:

- RAG systems still make up answers

- There is no way to detect made-up answers that is also efficient
- We rely much on big Large Language Models(LLM)
- It costs much and takes too long to compute
- It is not suitable for real-time applications.

This research is about creating a LLM-based system to detect made-up answers that can be used with RAG systems to make them more reliable and efficient. The goal is to design and implement a LLM-based hallucination detection framework that can be integrated into RAG pipelines to improve reliability while maintaining efficiency. This framework aims to make RAG systems more reliable, by detecting responses. RAG systems are used in applications and a small LLM-based hallucination detection framework can be used to improve their reliability. The small LLM-based hallucination detection framework will help to ensure the faithfulness of generated responses.

1.3 Objectives

The main goals of this research are:

- To study hallucination problems in Retrieval-Augmented Generation(RAG) systems
- To find limitations in hallucination detection methods
- To create a simple hallucination detection model using small language models like BERT or DistilBERT [7, 10]
- To build a system that can label generated responses as reliable or hallucinated
- To test the performance of the proposed model using common metrics like accuracy and F1-score
- To compare how efficient the proposed approach is in terms of computing cost and inference time.

This research aims to solve the issue of fake outputs in RAG systems while reducing the need for large and costly models, which will make it more efficient and scalable.

The research focuses on hallucination issues in RAG systems and hallucination detection methods. It also aims to design a hallucination detection model using small language models such as BERT or DistilBERT to classify generated responses as faithful or hallucinated. The goal is to evaluate the performance of the proposed model and compare its efficiency.

The research is about hallucination, in RAG systems and improving efficiency.

1.4 Methodology

This study is aimed at designing a method to detect hallucinations in a particular system. In order to develop and test the developed framework effectively, the following research is needed to design and create an efficient and resourceless framework for detecting hallucinations.

The proposed approach has a list of key activities as follows:

- **Data acquisition and preprocessing:** In order to examine the efficiency of the developed system, a dedicated dataset will be used, which will be represented by the RAGTruth corpus [16]. For further processing, the data will be divided into contexts-questions-answers sets. Further, in order to bring information to the required level and preserve its integrity, we will need to conduct pre-processing with the help of a specific tool.
- **Model selection and architectural design:** We will refuse to apply GPT-4 and Llama-3 models due to their resource requirements. As an alternative solution, a Small Language Model (SLM) will be applied, which will be represented by the DeBERTa and ModernBERT frameworks. They will analyze whether the answer's word is consistent with the context.
- **Training and Optimization:** Our Small Language Model will be trained using the data we already have. In order to ensure that it is efficient, we will use advanced methods such as Low-Rank Adaptation.
- **Evaluation:** The performance of our proposed approach will be tested. Such measures as F1-score and Precision will be used to evaluate the ability of the model to identify the errors.
- **Comparative Cost-Benefit Analysis:** Lastly, we will conduct a comparative study between our solution and that based on large models to determine the efficiency of our approach and its resource requirements.

The hallucination detection framework will be. Evaluated using the RAG hallucination evaluation dataset. Our Small Language Model will be applied to detect hallucinations in the system. Our approach will be designed in such a way to detect the hallucinations efficiently and without high resource consumption.

1.5 Organization of the Report

This report is divided into chapters like this:

- Chapter 2 talks about the project’s background. It covers Large Language Models(LLM), Retrieval-Augmented Generation problems with hallucination and a review of studies. It also looks at what’s missing in research.
- Chapter 3 explains how the system is designed. This includes figuring out what is needed planning the system’s structure, understanding how data moves through the system and designing the model.
- Chapter 4 goes into detail, about how the system was built and tested. It covers getting the data ready, training the model and checking how well it works.
- Chapter 5 discusses the challenges of designing the system, its limitations and how much it costs.

Finally Chapter 6 sums up the report. It also suggests what research should be done next and how the system could be improved.

Chapter 2 Background

Having considered all of the below, for our primary ideal of relating visions in RAG using small LLMs, we need an introductory understanding of the abecedarian generalities and to dissect the methodologies and ways, along with their architectural and evaluation approaches. Following an analysis of several papers and fabrics applicable to our study, we identify the algorithms and processes that show promise, as well as areas of exploration to concentrate on in our thesis. Let us have a look at the fundamentals involved in our exploration.

2.1 Preliminaries

This design would not be possible without a disquisition on a large scale in the field of Natural Language Processing(NLP) and Information Retrieval. After a mass analysis of papers related to our work, we can presume what algorithms and procedures we might follow. This section defines the foundational generalities, algorithms, and evaluation criteria that are central to our exploration.

2.1.1 Large Language Models(LLM)

The Large Language Models(LLM) are deep literacy models which are built on huge textbooks of data to comprehend, infer and control mortal languages. They are the basic "makers" in the framework of RAG. Although they are important, they are highly susceptible to illusions. In our experiment, we focus on two categories of models. We will first consider large models such as GPT-4 and Llama-2 to generate responses. The second category would be smaller models like ModernBERT [1], DeBERTa [2] and Baichuan2-7B [6] for sensor operations. The most important aspect of smaller models is their efficiency, minimal latency and cost-effective deployment in product environments.

2.1.2 Retrieval-Augmented Generation (RAG)

RAG is an approach that improves the performance of LLMs by giving them access to external and relevant knowledge sources. A common architecture for RAG includes two parts: a retriever module that retrieves relevant context documents (company policies, Wikipedia pages) from the knowledge base according to a user's input query, and a generator that takes the query and context as inputs to generate the response. By doing so, the generated response should contain facts that can be traced back to

verifiable sources, thus reducing intrinsic hallucinations. Nonetheless, as shown in our findings, RAG systems still suffer from extrinsic hallucinations that are inconsistent with the provided context.

2.1.3 Hallucinations in Large Language Models (LLMs)

In the case of hallucination within LLMs, this can be described as output that is not only irrelevant but also wrong or inconsistent with the provided context information. The academic categorization of hallucinations generally splits them into two main types:

Intrinsic Hallucination: The output is in conflict with the source context or user query (for example, "The color of the car is blue," when the text specifies "red").

Extrinsic Hallucination: The output is impossible to verify based on the source context.

Why use Small LLMs for this task?

Instead of using larger models like GPT-4, which has more than 1 trillion parameters, researchers use smaller LLMs that need less than 1 billion parameters to detect hallucinations [7, 10]. Common examples include: -

- ModernBERT (150M to 396M parameters) [1]
- DeBERTa-large (440M parameters) [2]
- Baichuan2-7B (7 billion parameters, but still counted as small LLMs) [6]

Small LLMs are specifically trained for the hallucination detection task, offering competitive accuracy and efficiency.

2.1.4 Approaches for Detecting Hallucinations

Approaches for identifying hallucinations may be classified based on their granularity and approach used:

Granularity: The detection could be at the example-level (where we classify the entire answer as either a hallucination or not), or at a more fine-grained span/sentence/token-level (where we determine the precise section of the answer where a hallucination occurred).

Approaches:

- **Fine-Tuned LLM Judges:** Training a specific LLM (which is usually much smaller than the source model) on an annotated corpus of examples with hallucination/non-hallucination labels [3,4]. This is the approach we are using in our work.

- **Encoder-Based Approach:** Employing smaller models with bidirectional encoding capabilities (BERT, DeBERTa, ModernBERT), which act as classifiers to evaluate the relation between the context and the answer, which may also be framed as a natural language inference problem [1, 2].

2.1.5 Key Algorithms and Architectures

- **Transformer Model & Attention Mechanism:** The foundation for current LLMs where the weights of each word in a sequence are calculated using the attention mechanism [1].
- **BERT (Bidirectional Encoder Representations from Transformers):** An initial encoder-only model where learning context is done by considering words from both left and right directions. The model has a limitation of dealing with sequences of up to 512 tokens only [1].
- **DeBERTa (Decoding-enhanced BERT with disentangled attention):** A refined BERT-like model with high performance on NLI problems that is able to detect hallucinations (e.g., Luna model). The model has been successful at solving the contradiction classification problem [2].
- **ModernBERT:** The cutting-edge encoder-only model where architectural advancements such as rotary positional embedding (RoPE) and alternating attention have been made. Its primary strength is its ability to process long contexts up to 8,192 tokens [1].
- **Natural Language Inference (NLI):** A task of classifying the relation between a "premise" (context) and a "hypothesis" (generated answer) as either entailment (supports), contradiction (disagrees) or neutral [2,6].

2.1.6 Evaluation Metrics

To measure the performance of our daydream discovery model, we will use the ensuing standard criteria[1,3].

Precision: of all the answers the model flagged as 'hallucinated', how numerous were actually visions?(Measures false cons).

Recall(perceptivity): Of all the factual visions in the dataset, how numerous did our model successfully catch?(Measures false negatives).

F1- Score: The harmonious mean of Precision and Recall, furnishing a single, balanced metric for model performance. This is the primary metric used in our birth comparisons.

Delicacy: The overall chance of correct prognostications(both visions andnon-hallucinations rightly linked).

Area Under the ROC wind(AUROC): A measure of a model’s capability to distinguish between positive(hallucinated) and negative(non-hallucinated) classes across different decision thresholds. An advanced AUROC indicates better demarcation.

2.2 Literature Review

The work is positioned at the crossroads between RAG models and effective hallucination detection techniques. The review presents an overview of prior works on related problems and, most significantly, scientific publications that have influenced the development of the domain.

2.2.1 Similar Applications (Evaluation Frameworks)

However, many frameworks created for testing RAG systems and identifying hallucinations utilize time-consuming and expensive approaches based on large models or their APIs.

- RAGAS (Retrieval Augmented Generation Assessment): An example of a framework that can be used for free reference-based evaluation [3]. Uses prompt-based LLMs (GPT-3.5) to calculate different scores such as Faithfulness, Answer Relevancy, and Context Relevance. Although useful, it uses LLMs’ APIs, thus increasing the cost and delay [3].
- Trulens: A framework similar to RAGAS that uses judges based on LLMs for evaluation metrics, including Groundedness. The real-time application of this framework is limited due to the performance of the underlying LLM [1].
- Vectara’s HHEM (Hughes Hallucination Evaluation Model): A specially fine-tuned version of a Transformer-based model to detect factual discrepancies [4,5]. In comparison with other solutions, it is relatively fast; however, it is a closed-source solution.

2.2.2 Related Research Papers

Our research is based on a detailed study of the following key papers, which are classified for ease of reference.

A. Foundational Encoder-based Detectors

- Luna (Belyi et al., 2025): The closest relative of our proposed research is the Luna paper, presenting a compact model for detecting hallucinations [2]. Luna

refers to a 440M parameter DeBERTa-large encoder trained for NLI tasks using RAG data. Luna introduces the concept of "precise context chunking," which prevents false positives due to large contexts. Luna will be the main baseline in the paper, which will present the better LettuceDetect model in terms of the F1 score[2].

- LettuceDetect (Kovacs & Recski, 2025): This paper significantly improves on Luna by using ModernBERT instead of DeBERTa [1]. By employing the long-context abilities of ModernBERT (8k tokens), it obtains an F1 score of 79.22% on the RAGTruth dataset, which is 14.8% higher than that of Luna. Importantly, this model is only 30 times smaller compared to the best existing language models [1].

B. LLM-based and Agentic Detectors

- RAGTruth (Niu et al., 2024): Not only does it serve as a detection algorithm, but also this paper presents the first benchmark on evaluating RAG hallucination. Specifically, it includes 18K human-labeled instances with respect to the QA, data-to-text generation, and summarization tasks [16]. Moreover, it offers the fine-tuned Llama-2-13B model as a strong baseline model. This work and other works utilize RAGTruth for evaluation purposes.
- HaluAgent (Cheng et al., 2024): This paper studies another extreme point by considering the task of autonomous agents in the detection of hallucinations through a tiny language model, i.e., Baichuan2-7B [6]. HaluAgent adopts a toolkit (e.g., search engine, calculator) along with a three-step framework to detect various types of hallucination. The results indicate that the tiny language model becomes capable of obtaining a high-performance level similar to GPT-4 after being fine-tuned [6].

C. Frameworks and Comparative Studies

- RAGAs (Es et al., 2024): In section 2.2.1, this paper defines a range of reference-free metrics for evaluating RAGs [3], presenting the ideas of Faithfulness, Answer Relevancy, and Context Relevance. It offers a valuable structure for comprehending the complex aspects of RAG quality.
- "Real-Time Assessment Models for RAG" (Sardana, 2024): This industry standard evaluates different real-time assessment models (LLM-as-a-Judge, HHEM, Prometheus, Lynx, TLM) [4] over several datasets (FinQA, ELI5, PubmedQA).

A significant discovery is that no one method overwhelmingly prevails, and approaches such as TLM (Trustworthy Language Model), which integrate self-reflection and consistency, frequently yield the best results. This supports our assumption that a mix of signals is advantageous [4].

2.2.3 Gap Analysis

Here, we discuss the gaps in different papers.

Table 2.1: Gap Analysis

Serial	Paper	Gap
1	[2]	Current encoder-based models such as Luna (using DeBERTa) face difficulties with lengthy RAG contexts (over 16k tokens) and necessitate intricate chunking methods that may lead to false positives when relevant data is dispersed across chunks.
2	[1]	Numerous cutting-edge detectors are either proprietary or depend on paid APIs (GPT-4) or have restrictive licenses, hindering their use in academic research and commercial settings.
3	[4]	Frameworks such as RAGAS and Trulens offer various metrics, yet their effectiveness in hallucination detection (AUROC) typically falls short when compared to dedicated detectors.
4	[6]	Absence of bilingual illusion identification abilities. The majority of current datasets and detectors are exclusively in English, offering little support for other languages such as Chinese.
5	[5]	Current benchmarks emphasize general knowledge or open-domain tasks, lacking standardized datasets to assess hallucinations in HR or corporate internal chatbot applications.
6	[3]	Achieving consistent outcomes with LLM-based evaluations is difficult because of undocumented API modifications and intrinsic randomness.

Chapter 3 Project Design

In this chapter, the architecture of the proposed SLM-based hallucination detection tool, along with the engineering process, is described. It will be covered such topics as the requirement analysis, the data flow, methodology, as well as task division within the project.

3.1 Requirement Analysis

A thorough requirement analysis was performed in order to ensure that the created system would be both efficient and extremely accurate.

3.1.1 Functional and Nonfunctional Requirements

Functional Requirements:

- **Input Ingestion:** The system should take three different types of input texts: Context, which is retrieved from an external source, User Question and LLM Answer.
- **Tokenization:** The system needs to be able to perform tokenization of input texts using the corresponding tokenizer of the selected SLM (DeBERTa/ModernBERT). Context windows should be handled correctly.
- **Token-Level Classification:** The SLM should give a probability score to each token from the answer generated by LLM indicating whether it is factually consistent with the given context.
- **Span Aggregation:** The system needs to aggregate hallucinated tokens that appear one after another in order to produce spans or sentences that show exact errors to the user.

Non-functional Requirements:

- **Low Latency (Performance):** The system needs to be able to do inference in near real-time (within 100-200 milliseconds for each inference) to be viable in real-life RAG applications.
- **Lightweight Deployment (Efficiency):** The system's core model should have no more than 500 million parameters to enable local deployment on one GPU/CPU device and eliminate reliance on third-party models.

- **High Accuracy (Reliability):** The system should exhibit high F1-score (target >75%) on the benchmarking dataset such as RAGTruth in order to have a low false positive rate.

3.1.2 Context Diagram

Central Node: "SLM Hallucination Detection Framework."

- **External Entity 1:** "RAG Pipeline/User" sends a packet of data with [Context+Question+Answer] to the central node.
- **External Entity 2:** The central node sends a response with [Verified Text+Highlighted Hallucinations+Confidence Score] to the "RAG Pipeline /User."

3.1.3 Data Flow Diagram

1. **Process 1.0 (Data Formatting):** Accepts the input data in their natural language form and converts them into Context-Question-Answer triples.
2. **Process 2.0 (Tokenization):** Converts the processed text into tensors through a tokenizer particular to the SLM.
3. **Process 3.0 (Inference Engine):** Feeds the tensors into the encoder-based SLM (like ModernBERT) for binary token classification as 0 for supported and 1 for hallucinated.
4. **Process 4.0 (Post-Processing):** Sets a confidence threshold value (>0.5) for the probability of the tokens and summarizes them into highlighted text segments.
5. **Data Store (D1):** A locally stored component recording evaluation metrics and errors to fine-tune the system later.

3.1.4 Design

Software Stack (The Tools)

- **Programming Language:**
 - Python (3.9 or higher): Industry-standard programming language used in the field of machine learning and AI.
- **Core Machine Learning Framework:**
 - PyTorch: Framework you will mainly be using to load and fine-tune your ML model. PyTorch has great compatibility with Hugging Face.

- Model & Tokenization Libraries:
 - Hugging Face transformers: Library used to load the pretrained models (ModernBERT [1] and DeBERTa [2]).
 - Hugging Face tokenizers: Used to turn your Context-Question-Answer text into tensors for your model.
 - Hugging Face datasets: The fastest way to download, manipulate, and process your dataset (RAGTruth dataset [16]).
- Fine-Tuning Libraries:
 - Hugging Face peft (Parameter-Efficient Fine-Tuning): Necessary library to apply LoRA, meaning that you do not require a powerful GPU in order to train your model.
 - bitsandbytes (Optional but recommended): Needed if you wish to run the model on consumer-grade hardware by applying quantization (8-bit/4-bit precision)
- Evaluation & Metrics:
 - scikit-learn: Library useful for measuring performance on classification tasks via F1-score, Precision, Recall and AUROC measures.
- Deployment & User Interface:
 - FastAPI: Web framework used to deploy an ML model via the backend API.
 - Streamlit: App Framework for building the web dashboard for users to easily provide their Context-Question-Answer text.
 - Uvicorn: ASGI (Asynchronous Server Gateway Interface) used to host your FastAPI application.

Hardware Stack (The Computing Power)

- For Model Training & Fine-Tuning (Heavy Workload):
 - Option A (Cloud - Recommended): Google Colab Pro or Kaggle. Google Colab Pro or Kaggle. To train the LoRA adapters, you'll require access to at least one T4 GPU (16GB VRAM) or A100 GPU (40GB VRAM) for some time.
 - Option B (Local): An advanced desktop computer or a gaming computer with NVIDIA GPU. It is recommended to have an RTX 3090, 4080, or 4090 (minimum 16-24GB of VRAM) to train your 100M-400M parameter models locally.

- For Inference & UI Deployment (Light Workload):
 - Local PC / Laptop: After training and quantizing the model, it becomes really light in terms of memory usage. The inference API and Streamlit UI can be hosted on any average laptop computer with a modern processor (Intel i7/i9, AMD Ryzen, Apple M-series) and 16GB system RAM. The use of local GPU will give instant results in milliseconds, while a simple CPU would also do the trick for demonstration purposes.

3.2 Detailed Methodology and Design

The design moves down from erecting resource-intensive systems like 'LLM-as-a-Judge' [3, 4] and adopts a more doable approach using Natural Language Inference (NLI) and commemorative bracket ways to estimate affair correctness, particularly for relating visions.

Indispensable results Considered, Generative LLMs (Llama-3-8B, GPT- 3.5) were estimated for zero- shot and prompt- grounded double judgment (' True/ False') but barred due to inordinate computing coffers needed and implicit performance backups compared to reclamation- stoked generation (RAG) systems. A featherlight commemorative- bracket model was also explored as an intermediate but was supposed to be infelicitous since it fails to deliver the necessary grainy explanations needed for the task.

Named the architecture, we named a token-position bracket armature powered by an encoder-only SLM (similar to DeBERTa-large [2] or ModernBERT [1]). This system enables the system to effectively understand the environment from both directions. It combines the environment, question, and answer, using attention mechanisms to ensure the generated answer aligns well with the source environment. By prognosticating chances at the token position, the system can pinpoint inaccuracies with great precision. Also, to conform the model for specific datasets like RAGTruth while staying within tackle constraints, we'll use Parameter-Effective Fine- Tuning (PEFT) with Low-Rank adaptation (LoRA) [10].

3.3 Project Plan

The design cycle has been divided into four major phases throughout the academic trimesters.

- Phase 1 - Research & Data Preparation (Weeks 1- 3): Literature check, system architecture description, and accession/ preprocessing of the RAGTruth standard dataset.

- Phase 2 - Model Development & Training(Weeks 4- 7): Tokenizer execution, SLM architecture configuration, model fine- tuning using LoRA.
- Phase 3 - Evaluation & UI Integration(Weeks 8- 10): Assess the model in terms of standard criteria(F1- Score, AUROC), establish a birth quiescence, and integrate a conclusion machine via the web UI.
- Phase 4 - Documentation(Weeks 11- 12): Composing the design report, developing the visualizations, and organizing the final defense donation.

3.4 Task Allocation

Table 3.1: Task Allocation (SLM Hallucination Detector – Group B2)

ID	Task	Assigned To	Progress	Start	End	Days
Phase 1: Research & Dataset Preperation						
1.1	Literature Review & Problem Definition	Mehrin	100%	4/1/26	4/14/26	14
1.2	Dataset Acquisition (RAGTruth)	Mehrin	100%	4/1/26	4/7/26	7
1.3	Preprocessing RAGTruth Data	Mehrin	100%	4/7/26	4/21/26	14
1.4	Selected SLM Environ-ment Setup	Mominul	100%	4/14/26	4/28/26	14
1.5	Define Fine-tuning Pipeline	Mominul & Mainul	100%	4/21/26	5/6/26	14
Phase 2: Model Development & Fine-tuning						
2.1	Implement Token-level classifier Architecture	Mominul	80%	5/7/26	5/28/26	21
2.2	PEFT/LoRA Integra-tion & Testing	Mominul	70%	5/14/26	5/28/26	14
2.3	Develop Evaluation Module (F1, AUROC)	Mehrin & Mainul	60%	5/14/26	6/4/26	21
2.4	Fine-tuning on RAGTruth Dataset	Mominul & Mehryn	40%	5/28/26	6/11/26	14
2.5	Initial Results Analysis	All	20%	4/11/26	6/25/26	14

Table 3.1: Task Allocation (SLM Hallucination Detector
– Group B2)

ID	Task	Assigned To	Progress	Start	End	Days
2.6	UI Mockup & Integration Plan	Nazmul	90%	5/1/26	5/15/26	14

Project Leader: Md. Mainul Hossain Fahim Chowdhury
Project Start: Wed, 4/1/2026
Today: Mon, 6/1/2026

3.5 Summary

This chapter presented the step-by-step engineering process for developing an effective hallucination detector framework. Through the articulation of specific functional requirements and understanding of how the data flow from inputted raw text to highlighting outputs in the user interface can be achieved through the SLM classifier at the token level, an effective blueprint has been formed. Efficient distribution of responsibilities ensures the smooth completion of the remaining processes within this project.

Chapter 5 Standards and Design Constraints

This chapter details the complex engineering problems and conditioning addressed by our featherlight daydream discovery frame. It also outlines the critical design constraints ranging from profitable feasibility to computational sustainability that guided the development of this system.

5.1 Complex Engineering Problems Activities

To make a better nation, masterminds need to concentrate on large- scale development through processes that break critical problems of our society and by which a nation might be served. According to the " Washington Accord," there are specific criteria that need to be reckoned for to meet the description of complex engineering problems and conditioning.

5.1.1 P's that are addressed through our project

- P1(Depth of Knowledge): Developing this system requires in- depth knowledge of Natural Language Processing(NLP) and ultramodern motor infrastructures(K3, K4). It demands a rigorous understanding of Natural Language Inference(NLI), token- position bracket mathematics, and the operation of Parameter-Effective Fine- Tuning(LEFT) ways like Lora to modify small language models similar as Modernist and DeBERTa(K5, K6) [1, 2, 10].
- P2(Clashing Conditions): The design must balance two clashing engineering conditions achieving high evaluation delicacy(F1- score) while maintaining extremely low computational quiescence and memory operation. Traditional large models offer delicacy but warrant effectiveness, forcing us to introduce with SLMs to satisfy both constraints.
- P3(Depth of Analysis): Rigorous logical capability is needed to reuse standard datasets(like RAG Truth) and interpret token- position probability scores. The system must analytically separate between an inadequately stated answer and an authentically hallucinated fact to minimize false positive rates.
- P4(Familiarity of Issues): We must be deeply familiar with the well- proved vulnerabilities of Retrieval- Augmented Generation(RAG) channels — specifically foreign visions and the severe quiescence issues associated with current " LLM- as-a-judge" evaluation fabrics(like RAGAS or Rules) [3, 4].

- P6(Extent of Stakeholder Involvement): This frame is designed for different stakeholders, including software masterminds integrating RAG into operations, enterprises taking strict data sequestration (original prosecution), and end-users who calculate on factual, secure AI responses.
- P7(Interdependence): This problem can not be answered in isolation. The system requires the flawless interdependence of data preprocessing channels, advanced tokenizers, an SLM conclusion machine, span- aggregation algorithms, and a stoner-facing web dashboard.

Table 5.1: Mapping with complex problem solving.

P1	P2	P3	P4
Dept of Knowledge	Range of Conflicting Requirements	Depth of Analysis	Familiarity of Issue
Addressed			
✓	✓	✓	✓

Table 5.2: Project Cost Analysis

Serial No.	Item / Component	Description	Quantity	Unit Cost (Dollar)	Total
1	Development Hardware	Standard Laptops for coding, UI development, and lightweight inference.	4	Existing	
2	Cloud Compute (GPU)	Google Colab Pro subscription for intensive LoRA fine-tuning (Estimated 2 months).	2	~\$10	
3	Software Frameworks	Python, PyTorch, FastAPI, Streamlit, and Hugging Face libraries.	N/A	Open Source	
4	SLM Architectures	Pre-trained base models (ModernBERT, DeBERTa-large).	2	Open Weights	
5	Benchmark Datasets	RAGTruth corpus for training and evaluation.	1	Open Data	
6	Deployment / Hosting	Localhost execution and Streamlit Community Cloud for web UI demonstration.	1	Free Tier	
7	Documentation	FYDP Report printing, binding, and presentation materials.	3	~\$5	
Total Estimated Cost:					

Cost Justification:

The estimated cost for this project is very low around ~\$35 .

If we used an approach with OpenAI API, like GPT-4, the costs for checking a dataset like RAGTruth would be very high—hundreds of dollars.

Our SLM architecture makes these API costs go away making it affordable for research and business use.

This project can save money with our SLM-based approach.

The traditional "LLM-as-a-judge" approach is costly because of API fees.

Our method is cheaper. Works well, for academic and enterprise use.

5.4 Summary

This part of the project talks about the engineering problems we are trying to fix. We are using our knowledge of NLP to solve problems with RAG. We are being very careful about money and the environment and ethics. So the SLM framework we are making is not just very accurate, but very sustainable and private and can be used on regular consumer hardware, like the devices people have at home.

Bibliography

- [1] Á. Kovács and G. Recski, "LettuceDetect: A Hallucination Detection Framework for RAG Applications," arXiv preprint arXiv:2502.17125, 2024.
- [2] M. Belyi, R. Friel, S. Shao, and A. Sanyal, "Luna: A Lightweight Evaluation Model to Catch Language Model Hallucinations with High Accuracy and Low Cost," arXiv preprint arXiv:2406.04523, 2024.
- [3] S. Es, J. James, L. E. Anke, and S. Schockaert, "RAGAS: Automated Evaluation of Retrieval Augmented Generation," in Proc. EACL (System Demonstrations), 2024, pp. 150-158.
- [4] A. Sardana, "Real-Time Evaluation Models for RAG: Who Detects Hallucinations Best?" Towards Data Science, 2024.
- [5] R. Bansal et al., "Understanding and Mitigating Strategies for Large Language Model (LLMs) Hallucinations in HR Chatbots," Int. J. Intell. Syst. Appl., vol. 17, no. 1, pp. 45-60, 2025.
- [6] X. Cheng et al., "Small Agent Can Also Rock! Empowering Small Language Models as Hallucination Detector," arXiv preprint arXiv:2311.08642, 2023.
- [7] F. A. Senella and H. B. Ozmen, "A comparative review of hallucination mitigation and performance improvement techniques in Small Language Models," Research & Design, vol. 2, no. 1, pp. 45-65, 2025.
- [8] M. P. Priola, "Addressing Hallucinations with RAG and NMISS in Italian Healthcare LLM Chatbots," arXiv preprint arXiv:2412.04235v3, 2024.
- [9] A. Aman, "Detecting LLM Hallucinations: A Lightweight Type-Aware Hallucination Evaluation Model," M.S. dissertation, Dept. Informatics, Univ. Sussex, Summer 2025.
- [10] K.-C. Pai and W.-C. Hsu, "Fine-tuning small language models for industrial retrieval-augmented generation: Efficiency, factuality, and hallucination analysis," Computer Standards & Interfaces, vol. 98, p. 104163, 2026.
- [11] X. Chen, Q. Tang, and L. Wang, "Honest AI: Fine-Tuning 'Small' Language Models to Say 'I Don't Know', and Reducing Hallucination in RAG," arXiv preprint arXiv:2410.09699, 2024.

-
- [12] Y. F. Bezerra and L. Weigang, "LLMQuoter: Enhancing RAG Capabilities Through Efficient Quote Extraction From Large Contexts," arXiv preprint arXiv:2501.05554v1, 2025.
 - [13] P. Béchard and O. M. Ayala, "Reducing hallucination in structured outputs via Retrieval-Augmented Generation," ServiceNow Research.
 - [14] P. Manakul, A. Liusie, and M. J. F. Gales, "SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models," ALTA Institute, Department of Engineering, University of Cambridge.
 - [15] F. Ridder and M. Schilling, "The HalluRAG Dataset: Detecting Closed-Domain Hallucinations in RAG Applications Using an LLM's Internal States," arXiv preprint arXiv:2412.17056v2, 2025.
 - [16] C. Niu, Y. Wu, J. Zhu, S. Xu, K. Shum, R. Zhong, J. Song, and T. Zhang, "RAGTruth: A Hallucination Corpus for Developing Trustworthy Retrieval-Augmented Language Models," arXiv preprint arXiv:2401.00396, 2024.